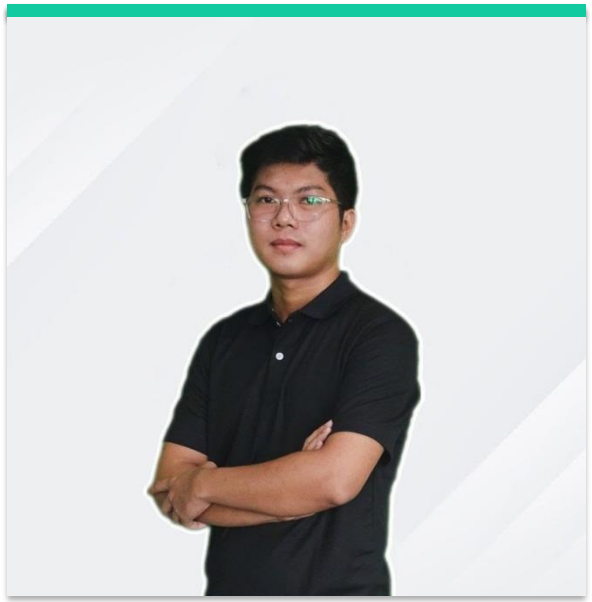
THE VIBE PARADIGM

Agentic AI & Vibe-Coding Training Series 2026

UNDP IICPSD — Technical Training Series

Presenter: Phyo, Senior Cyber Security Analyst

Meet Your Instructor



Phyo Ko Ko

Senior Cyber Security Analyst — Thailand

As a Senior Cyber Security Analyst based in Thailand, Phyo focuses on integrating Machine Learning and advanced AI workflows into high-stakes enterprise systems, incident response, and cloud security.

- 4+ years of professional cybersecurity experience
- Specialized ML/DL training with Stanford and UNDP
- Expertise: Multi-agent systems, Incident response, Threat Hunting, Vulnerability finding, Cloud Infrastructure
- Mission: equip tech workforces in LDCs with cutting-edge AI frameworks

What You Will Learn — 5 Modules

01

Markdown & Prompting

Structure prompts that control LLM attention and reduce hallucinations

02

RAG & Context

Retrieve only the relevant code using semantic search and grounding

03

Security & Safety Net

Build automated guardrails that catch vulnerabilities before production

04

MCP & Multi-Agent

Connect agents to live tools and orchestrate multi-agent pipelines

05

Capstone Challenge

Run a full vibe coding pipeline from prompt to production-ready code

The Strategic Opportunity — Why This Matters for LDCs

The Traditional Barrier

- ✕ Required 5–10+ years of syntax mastery to build production software
- ✕ Expensive senior developers concentrated in wealthy nations
- ✕ Months to go from idea to working product
- ✕ LDC tech workforces excluded from high-value software markets
- ✕ Global digital divide keeps widening

The Vibe Advantage

- ✓ Natural language replaces syntax — describe intent, AI writes the code
- ✓ LDC teams build enterprise-grade solutions immediately
- ✓ Time-to-market compressed from months to hours
- ✓ Global participation at unprecedented speed and scale
- ✓ A junior developer can orchestrate like a senior architect

The Trust Advantage: Speed is now universal. The competitive edge for LDC tech workforces is proving that rapidly generated agentic code is secure, scalable, and enterprise-grade.

What is Vibe Coding?

The Old Paradigm

- ✕ Manual syntax typing — brackets, semicolons, language rules
- ✕ Searching documentation for every edge case
- ✕ Months of learning a framework before being productive
- ✕ Developer defines WHAT and HOW — writes every line
- ✕ Speed limited by human typing and memorization

The Vibe Paradigm

- ✓ Natural language orchestration — describe intent, AI writes the syntax
- ✓ AI reads your full codebase and suggests context-aware solutions
- ✓ Build sophisticated features in hours, not weeks
- ✓ Developer defines WHAT — AI writes the HOW
- ✓ Speed limited only by the quality of your prompt structure

"The hottest new programming language is English." — Andrej Karpathy

The Evolution of AI-Assisted Development

PHASE 1

2021 – 2022

Autocomplete

Predictive inline suggestions. AI completes the next line based on context. No understanding of full project architecture.

PHASE 2

2022 – 2024

Prompt Engineering

Chat assistants sitting outside the codebase. Developers copy-paste code back and forth. No direct file access.

PHASE 3

2024 – Present

Agentic Workflows

Autonomous agents that read entire codebases, create files, execute terminal commands, run tests, and self-debug.

The Vibe Coding Software Ecosystem

AI-Native IDEs (Gold Standard)

Cursor

Built natively for agentic engineering. Project-wide codebase indexing, .cursorrules rule enforcement, and multi-file context awareness out of the box.

Windsurf

Driven by the "Flow" paradigm — continuous autonomous agent execution alongside the developer. Minimises context-switching and manual intervention.

Traditional IDE + Agentic Extensions

VS Code + Cline

Open-source agentic layer over VS Code. Full file-system access, terminal execution, and multi-model support.

VS Code + Roo Code

Alternative agentic extension with boomerang task orchestration and custom model routing.

The Engines vs. The Interfaces: Cursor and Windsurf are the shell you work inside. The intelligence comes from the foundation model: Claude Sonnet/Opus, GPT-4o, or Gemini. Selecting the right model for the task is itself a core professional skill.

MODULE 01

Markdown & Structured Prompting

The native language of AI — how structure controls LLM attention

The Native Language of AI — Markdown

Semantic Attention Weights

Headers (#, ##, ###) act as structural nodes that orient the LLM's attention mechanism. Content under a header receives significantly more attention than unstructured prose.

Boundary Isolation

Triple-backtick code blocks (```) tell the model exactly where data literals end and actionable instructions begin. Without this, the LLM blurs data and logic.

Bullet Hierarchy

Nested bullets create explicit parent-child relationships that help the model understand priority and sequence. Unstructured text produces flat generation with higher hallucination rates.

Native Thought Language

LLMs are trained overwhelmingly on Markdown-formatted documents (GitHub, Stack Overflow, documentation). Markdown is statistically the highest-signal format for structured reasoning.

Prompt Engineering & API FinOps

Context Caching

Structure static rules and documentation at the TOP of the context window to leverage lower-cost cached tokens. Cached tokens can be 10x cheaper than uncached input tokens.

Token Bloat Reduction

Keep multi-file prompts specific and structured via Markdown. Injecting an entire codebase when only one function is relevant wastes 80–90% of your API budget.

Asymmetric Model Layering

Use smaller faster models (Haiku, GPT-4o-mini) for boilerplate. Reserve high-tier engines (Claude Sonnet, o3) purely for architecture and security auditing.

The financial reality: blindly throwing large context windows at expensive models creates unsustainable cloud API costs at enterprise scale.

RAG & Codebase Context

Retrieve only the relevant code using semantic search — ground your agent in reality

How an Agent ‘Sees’ Your Codebase — RAG

1

Indexing

The IDE scans your project files and creates vector embeddings of every function, class, and comment.

2

Query

Your prompt is converted to a vector embedding in the same semantic space as your codebase.

3

Retrieval

Cosine similarity search finds the most semantically relevant code snippets to your intent.

4

Injection

Only those relevant snippets are injected into the hidden background context window before the model sees your prompt.

When you write a prompt, it is also converted to a vector embedding. The RAG engine finds code that is semantically similar to your intent — this is why Cursor “knows” your codebase without you telling it which file to read.

Context Engineering — Grounding the Agent

IDE Context Controls

@Files

Pin specific files — ensures the agent sees exactly the module it needs to modify.

@Folders

Include a full directory — injects all data models or API routes into context.

@Docs

Reference external docs — so the agent uses the correct library API, not a hallucinated one.

.cursorrules

Project-level security rules injected into every prompt automatically.

Why This Matters

Without explicit context grounding, the LLM generates code in a vacuum — it invents APIs, mismatches schema, and ignores your project structure.

With precise @File and @Folder references, the agent operates on your actual codebase, not a hallucinated approximation of it.

Context engineering is the difference between a junior developer who guesses and a senior one who reads the spec before coding.

The 4-Layer Context Model — What the Agent Actually Receives

Layer 1 — System Prompt

Role, rules, tone, output format. Set once per session.

e.g. "You are a senior security engineer. Flag hardcoded credentials. Output a corrected function."

Layer 2 — Retrieved Context (RAG)

Relevant files injected by the IDE or agent via semantic search.

e.g. "--- auth.py ---\ndef login_user(username, password): …"

Layer 3 — Conversation History

Prior turns kept in the context window so the agent remembers decisions.

e.g. "User: harden auth module → Agent: I'll review credentials, rate limiting, and input validation."

Layer 4 — User Message

The actual instruction for this turn.

e.g. "Now add rate limiting to prevent brute-force attacks."

assembled → LLM

Context Quality > Model Quality

The gap between a good AI response and a hallucinated one is almost never the model.

It is the quality of the context assembled around the instruction.

Context engineering is the discipline of closing that gap systematically.

Token Budget Reality

Layers 1-3 cost tokens every call.

RAG (Layer 2) keeps cost controlled — inject only the relevant files, not the whole codebase.

At enterprise scale, this is a FinOps decision.

Security & The Safety Net

Speed is a commodity — trust is the competitive advantage

The “Vibe Coding” Reality

The Opportunity

- ✓ Agentic workflows allow LDC teams to build complex features at unprecedented speed
- ✓ Eliminates years of syntax memorization — focus on systems thinking
- ✓ Enables global participation in software development at scale
- ✓ Reduces time-to-market from weeks to hours

The Risk

- ⚠ Speed without validation creates a backdoor into your systems
- ⚠ Research shows that 45% of AI-generated code contains at least one vulnerability
- ⚠ Hardcoded secrets, SQL injection, and eval() appear in AI output regularly
- ⚠ A single missed credential in source control can compromise an entire org

“Naked” vibe coding — accepting AI output without validation — leads to systemic technical debt and enterprise-grade liability.

THE SOLUTION

Architecting the Automated Guardrails

Pillar 1

Static Guardrails

Block vulnerabilities at code generation time — before the LLM even writes a line. Rules, pre-commit hooks, secret scanning.

Pillar 2

Dynamic Validation

AI writes and runs its own unit tests — catches runtime logic errors, edge cases, and integration failures automatically.

Pillar 3

Model-Graded Evals

Deploy a separate high-reasoning LLM to independently audit the code output for security compliance and architectural quality.

Pillar 1: Static Guardrails

Rule Enforcement — .cursorrules

Bake security rules into the agent's context before a single line of code is written. These rules are injected into every prompt automatically.

- ✓ Never hardcode API keys, passwords, or tokens. Always use environment variables.
- ✓ Always sanitize user input before passing to any database query. Use parameterized queries only.
- ✓ Never use eval() or exec() with unsanitized input.
- ✓ Always validate file paths to prevent directory traversal attacks.

Pre-Commit Hooks

Scripts that run automatically before a git commit is allowed to proceed.

TruffleHog

Scans recursively for leaked secrets, API keys, and credentials before they reach source control.

Bandit

Runs AST-based Python security analysis to flag dangerous patterns like eval() and pickle.

Semgrep

Language-agnostic static analysis — catches XSS, injection, and insecure deserialization.

Pillar 1.1 — Scanning Tools Deep Dive

TruffleHog

Secret Detection

- Scans git history recursively for API keys, tokens, and credentials
- Detects 700+ secret types across major cloud providers
- Integrates as a pre-commit hook — blocks commits before secrets reach source control
- Flags entropy-based anomalies even for custom secret formats

Bandit / Semgrep

Static Code Analysis

- Bandit: AST-based Python scanner (eval, pickle, subprocess risks)
- Semgrep: language-agnostic pattern matching for XSS, SQL injection, path traversal
- Both run in under 10 seconds on most codebases as CI gate checks
- Semgrep community rules updated continuously — catches newly discovered patterns

Snyk

Supply Chain Security

- Audits every third-party library the AI pulls into your project
- Blocks known CVEs and malicious packages before they enter your dependency tree
- AI agents often import unvetted packages — Snyk catches backdoors automatically
- Generates fix PRs automatically for vulnerable dependency upgrades

Pillar 2: Dynamic Validation — Agentic Unit Testing

Managerial QA — Not Manual QA

Instead of manually writing tests, instruct the agent to write and execute its own test suite immediately after generating code. Your role is to review test results, not write the tests.

Step 1

Instruct

Prompt: 'Write unit tests covering all edge cases and error paths for the code you just generated.'

Step 2

Generate

Agent writes pytest / Jest test file with 80%+ code coverage targeting boundary conditions and security cases.

Step 3

Execute

Agent runs the tests in the terminal via MCP. Any failure is automatically fed back into the context window.

Step 4

Self-Debug

Agent reads the failure traceback and rewrites the logic. Loop continues until all tests pass.

Pillar 3: Model-Graded Evals — The AI Auditor

The LLM-as-a-Judge Paradigm

Deploy a separate high-reasoning model (Claude Opus, o3) to independently audit the code generated by your coding agent. The auditor has no context of the original prompt and reviews the output purely for compliance and quality.

Security Compliance

Does the code introduce any OWASP Top 10 vulnerabilities? Any unvalidated inputs, hardcoded values, or injection vectors?

Architectural Soundness

Does the code follow separation of concerns? Are there tight couplings or god objects that will create technical debt?

Error Handling

Are all failure paths explicitly handled? Does the code fail safely or does it expose internal state on error?

Secret Hygiene

Are all credentials loaded from environment variables? Is there any accidental logging of sensitive data?

The Safety Net — Measured Effectiveness

90%+

of hardcoded secrets caught

85%+

of OWASP Top 10 blocked

100%

supply chain audited

3x

faster security review cycle

Vulnerability Type	Without Safety Net	With Safety Net
Hardcoded secrets in source control	Common — found in 1 in 3 repos	✓ Caught by TruffleHog pre-commit
SQL injection in AI-generated queries	High risk — AI skips sanitization	✓ Caught by Semgrep static analysis
Outdated vulnerable dependency	Silent — no audit on AI imports	✓ Caught by Snyk on every install
Logic error in edge case	Deployed — discovered in production	✓ Caught by agentic unit tests
Insecure eval() usage	Compiles fine — runtime exploit	✓ Caught by Bandit AST scan
Architectural security flaw	Shipped — manual review too slow	✓ Caught by model-graded eval

MODULE 04

Model Context Protocol (MCP) & Multi-Agents

Connect your agent to the real world — live tools, live data, live systems

Breaking the AI Sandbox — Enter MCP

The Problem: AI in a Box

Traditional AI coding assistants are sandboxed inside their local workspace.

They cannot:

- ✕ Query a live production database
- ✕ Read from a remote filesystem
- ✕ Call a live external API
- ✕ Send a Slack or email notification
- ✕ Check real-time system health status

Every external connection required custom hardcoded integration code written by a human engineer.

The Solution: MCP

Model Context Protocol (MCP) is an open standard created by Anthropic.

Any AI agent connects to any external tool through a single standardized interface — without custom integration code.

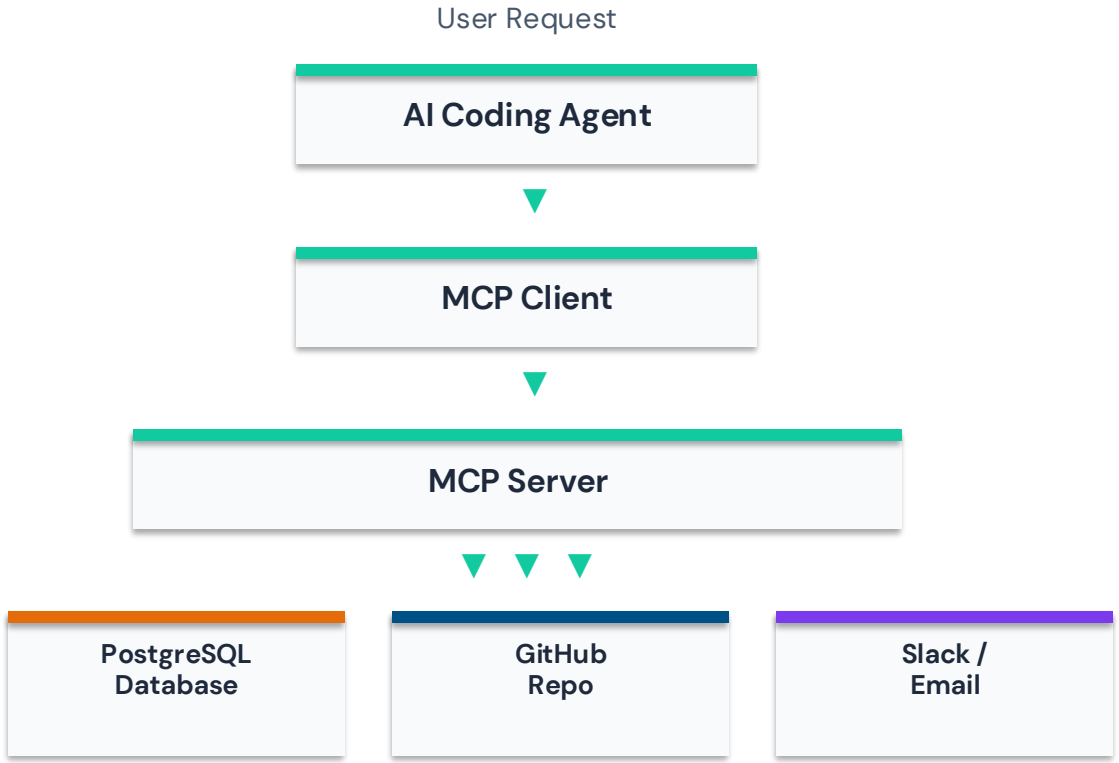
The USB Port Analogy:

Before USB: every device needed its own proprietary cable.

USB: one universal standard — any device, any computer.

MCP: one universal standard — any AI agent, any tool.

MCP Architecture — Server, Client, Tools



The agent connects to the server and reads the full tool list.

One MCP server can connect to: PostgreSQL, SharePoint, SAP, Slack, GitHub, and more.

Human approves each external action — the agent never acts autonomously on live systems without confirmation.

The Digital Workforce — Multi-Agent Pipelines

Agent 1

Product Owner

Reads the feature spec and breaks it into structured Markdown task files with clear acceptance criteria.

Agent 2

Coder

Ingests the task file, retrieves RAG context from the codebase, writes clean modular code.

Agent 3

Tester

Writes unit tests covering all edge cases. Executes them and feeds failures back into the pipeline.

Agent 4

Auditor

High-reasoning model reviews for security compliance, architecture quality, and code standards.

You

Orchestrator

Approve or reject each agent handoff. Define the values and acceptance criteria. You manage, not execute.

The Paradigm Shift: Replace a single overloaded developer with a coordinated AI assembly line. Each agent specialises. Each handoff is gated by human approval. Your role shifts from syntax writer to workflow architect.

MODULE 05

Capstone Challenge

End-to-end vibe coding pipeline — from prompt to production-ready code

Capstone — The 4-Step Vibe Coding Pipeline

Scenario: Build a user profile lookup feature that queries the database by username and returns email and role.

Step 1

Structure the Prompt

Write a Markdown-formatted prompt with clear headers, acceptance criteria, and data schema. No ambiguity.

Step 2

Ground with Context

Use @Files to pin your database models and API routes. Use @Docs for the ORM documentation.

Step 3

Generate & Validate

Agent writes the feature. Immediately instruct it to write unit tests and run them. Review only the test report.

Step 4

Audit the Output

Run TruffleHog + Bandit. Pass the final code to a model-graded eval for security review before committing.

All dependencies are pre-loaded in the Capstone setup cell — run it once and every helper function is available throughout the notebook.

Beyond the IDE — Enterprise Agentic Workflows

The Agent Roster

Agent 1: Product Owner

Breaks feature specs into structured Markdown task files.

Agent 2: Coder

Ingests the task file, retrieves RAG context, writes modular code.

Agent 3: Tester

Writes unit tests, runs them, reports failures back.

Agent 4: Auditor

High-reasoning LLM performs security compliance review.

You: Orchestrator

Approve or reject each agent handoff. Define values, not syntax.

Enterprise Frameworks

Azure AI Foundry

End-to-end multi-agent orchestration with enterprise access controls and audit logging.

Logic Apps

Event-driven: agent ingests an alert, triages it, writes a patch, prompts human for approval.

AutoGen / LangGraph

Open-source frameworks for stateful multi-agent pipelines with human-in-the-loop.

Claude API + MCP

Direct API integration: custom agents call any tool on your MCP server.

Beyond Web Apps — Infrastructure as Agent Territory

The Vibe Paradigm applies far beyond websites and APIs:

Cloud VPS Setup & Hardening

Agent scripts full server provisioning: user creation, SSH key config, UFW firewall rules, fail2ban, and automated security updates.

Network Routing & IP Tables

Agent writes iptables rules, manages port forwarding matrices, and configures routing for multi-VPS environments.

Encrypted Tunnel Deployment

Agent deploys Xray/V2Ray protocol tunnels. Handles certificate generation, config files, and systemd service setup.

Infrastructure as Code

Agent writes Terraform and Ansible playbooks from natural language specs. Validated in staging before production push.

The Engineering Workflow

1. Human describes architecture intent in plain language
2. Agent researches best practices via MCP + web search tools
3. Agent writes deployment scripts (bash, Terraform, Ansible)
4. Agent tests in staging — validates connectivity and config
5. Human reviews output and approves push to production

“Technology is a tool, but security is the foundation upon which trust is built.”

In the Agentic age, the manager’s role is not to write the code,
but to define the values that validate it.

The Action Item:

Transition immediately from syntax writer to enterprise systems auditor.

THANK YOU

Questions & Open Discussion

What questions do you have about integrating agentic AI into your work?

- All 5 Jupyter notebooks available in the course materials folder
- resource library.ipynb — curated videos, articles, and tool links by module
- Each module: concept explanation + working lab + graded assignment
- Start with Module 1 and build progressively through to the Capstone